Denotations for functions in which variables are first-class denotational citizens — or, "variables are data"

Murdoch J. Gabbay[1], Andrea Marin[2], and Samuel Rota Bulò[2]

[1] School of Computer Science
Heriot-Watt University
EDINBURGH EH14 4AS, Scotland
murdoch.gabbay@gmail.com
[2] Dipartimento di Informatica
Università Ca' Foscari di Venezia
Via Torino 155, 30172 Venezia Mestre, Italy
marin,srotabul@dsi.unive.it

Abstract. We promote a denotational semantics in which variables appear as first-class elements of the underlying (sets-based) domain. In this way new functions appear which intensionally examine variables and program on them as a form of data, with full access to the 'name' of a variable. At the same time, the extensional treatment of variables as representing an 'unknown element' is preserved.

1 Introduction

In this paper we promote a new semantics of variables in functional programming languages in which variables appear as concrete object in the underlying denotation, and can be programmed on as names — "the semantics gives access to the intension of a variable". For example, in our semantics there is a function $\equiv$ from numbers to booleans such that "$x \equiv y$" is *false*, because 'x' is not equal to 'y' (intensional equality); there is also a function $=$ such that '$x = y$' may be true or false, depending on what x and y are (extensional equality). The motivations for this work come from computer science and are discussed below.

Consider the typed λ-calculus and its semantics [5], which we now briefly construct: Choose a two-element set $o = \{\bot, \top\}$ and define types by $\tau ::= o \mid \tau \to \tau$ where $\to$ indicates the function-set. (It would be more usual to define a syntax for types and then map types to sets; for our purposes we can take types to *be* sets.) Fix $a, b, c, \ldots \in \mathbb{A}$ some countably infinite set of variable symbols or **atoms**; we use these throughout the paper. Terms are inductively defined by $t ::= a, b, c, \ldots \mid tt \mid \lambda a{:}\tau.t, \lambda b{:}\tau.t, \ldots$ quotiented by α-equivalence. A **typing environment** Σ maps atoms to types and a **typing judgement** is a triple $\Sigma \vdash t : \tau$. The **valid** typing judgements are inductively defined by:

$$\frac{}{\Sigma \vdash a : \Sigma(a)} \qquad \frac{\Sigma \vdash t_1 : \tau_2 \to \tau_1 \quad \Sigma \vdash t_2 : \tau_2}{\Sigma \vdash t_1 t_2 : \tau_1} \qquad \frac{\Sigma, a : \tau_2 \vdash t : \tau_1}{\Sigma \vdash \lambda a{:}\tau_2.t : \tau_2 \to \tau_1}$$

Here $\Sigma, a : \tau_2$ denotes the typing context such that $(\Sigma, a : \tau_2)(a) = \tau_2$ and $(\Sigma, a : \tau_2)(b) = \Sigma(b)$.

We can give terms a semantics as follows: if Σ is a type environment let a **valuation** be a map ς from a to $\varsigma(a) \in \Sigma(a)$. So ς maps atoms to *elements* of their types. Then ς extends to all terms by:

$$\varsigma(tt') = \varsigma(t)\varsigma(t') \qquad \varsigma(\lambda a{:}\tau_2.t) = \lambda x \in \tau_2.(\varsigma\{a \mapsto x\})(t).$$

Here $\varsigma\{a \mapsto x\}$ is the map such that $(\varsigma\{a \mapsto x\})(a) = x$ and $(\varsigma\{a \mapsto x\})(b) = \varsigma(b)$.

So far, so standard. As far as we know the idea of valuations is due to Tarski [42]. ML and Haskell [35, 43] are based on the typed λ-calculus with this semantics (Haskell more than ML since ML has global state, but never mind). They do all right; why should we be wanting any different?

As always, what we are happy to have depends on what we want to achieve. The semantics above is a useful abstraction for giving denotation to functional programming — but situations do exist which certainly *look* like functional programming, but where the semantics above is not the perfect abstraction:

- Suppose that we want patterns as first-class entities [3, 27, 11] so that we can pass them as explicit arguments to pattern-matching functions.
  Variables occur in patterns and the usual notion of variable must be modified in some way because the valuation ς identifies a variable a with its denotation $\varsigma(a)$. We expressly do *not* want this behaviour when we pass a pattern as an argument, because the variables in a pattern may be later matched to different things in different pattern-matchings.
- Suppose that we want to pass 'an explicit substitution' as a first-class entity $\lambda a.(a[b \mapsto 2])$ [20, 30, 40] such that $(\lambda a.(a[b \mapsto 2]))b$ equals 2. The valuation ς identifies a with $\varsigma(a)$ and b with $\varsigma(b)$. On the face of it $\varsigma(a)$ cannot 'mention a' or 'mention b', because a and b live in the meta-level (the syntax; the intension) and $\varsigma(a)$ and $\varsigma(b)$ live in the denotation (the semantics; the extension).
- Suppose that we want to examine an intensional property of a term at runtime, such as its free variables $\mathrm{fv}(t)$. This is not possible because the valuation ς identifies a with $\varsigma(a)$. Yet applications arise where this can be useful, for example if we wish to mix functional and logic programming by adding narrowing to β-reduction as a computational step [24] or in logic or programming frameworks where the predicate 'is a variable' is made explicit [21, 10].
- Now consider λ itself. There is no function in the usual semantics to denote a function λ-.- which 'takes a variable name a and an argument t and returns $\lambda a.t$'.
  More generally, it is not possible to give denotations to functions that bind variable names in their arguments, because variables (or to be quite precise, variable *names*) do *not* appear in the denotation. One example of such a function which we can imagine, but cannot easily give a denotation to, is λ-.- above; another is any 'context' such as $\lambda a.$-; the famous 'term with a hole' used for example in notions of program equivalence [37].

The references above are just some of many practical instances where the notion of variable name is made explicit in the language. These instances have not assembled themselves under a single denotational banner in anything like the way that functional programing languages are grouped around the banner of the 'standard' semantics described above (and its elaborations for practical systems, e.g. to account for global state in ML).

So this paper will attempt to create a 'sets and functions between them' semantics which exhibits by concrete construction certain ideas about how variables can be treated in denotation.

In discussions this material with colleagues we have noted a list of Frequently Asked Questions. We address them in the text with "$\mathcal{Q}$. *Question* $\mathcal{A}$. Answer" at strategic points.

Our tool in this programme is a notion due to the first author and Pitts [23]; this provides a syntax-free notion of 'element with some free names':

2 Nominal sets

A **permutation of atoms** π is a bijective function on $\mathbb{A}$ with **finite support**, meaning:

$$\text{For finitely many atoms } a \text{ we have } \pi(a) \neq a, \quad \text{and} \quad \pi(b) = b \text{ for all other atoms.}$$

Write $\mathbf{Id}$ for the **identity permutation** such that $\forall a.\mathbf{Id}(a) = a$. Write $(a\ b)$ for the **swapping** function such that $(a\ b)(a) = b$, $(a\ b)(b) = a$, and $(a\ b)x = x$ for all $x \neq a, b$ (note that $(a\ a)$ is the identity). Write functional composition of permutations as $\pi \circ \pi'$. Write $\mathbb{P}$ for the set of all permutations. It is a fact easily verified that swappings generate $\mathbb{P}$ as a group.

We use a convention that a, b, c vary *permutatively* over elements of $\mathbb{A}$ (unless otherwise stated). By this we mean that a and b are arbitrary *distinct* elements of $\mathbb{A}$. We adhere to this convention henceforth.

A **$\mathbb{P}$-action** on a set X is a function $(\mathbb{P} \times X) \to X$, sugar it to πx, such that $\mathbf{Id}x = x$ and $\pi(\pi' x) = (\pi \circ \pi')x$. Any $\mathbb{P}$-action is bijective on X; the inverse of π as a function on X is π^{-1}.

Call a pair $\mathbb{X}$ of a set X and a permutation action on it, a **set with a permutation action**. We may write $|\mathbb{X}|$ for X, or even just $\mathbb{X}$. We may write $x \in \mathbb{X}$ for $x \in |\mathbb{X}|$, and we may call $|\mathbb{X}|$ the **elements** of $\mathbb{X}$. We shall use similar conventions for substitution sets defined later. For example:

– $\mathbb{A}$ has the **natural permutation action** given by $\pi a = \pi(a)$.

– The set of finite lists of atoms $\mathbb{L}$, write its elements $[a_1, \ldots, a_n]$ (and write the empty list $[]$) has the **natural permutation action** given pointwise by $\pi[a_1, \ldots, a_n] = [\pi a_1, \ldots, \pi a_n]$.

– Call $U \subseteq \mathbb{A}$ **cofinite** when $\mathbb{A} \backslash U$ is finite. The set of finite and cofinite subsets of $\mathbb{A}$, write it $\mathcal{P}_{fs} = \{U \mid U \subseteq \mathbb{A},\ U \text{ finite or cofinite}\}$ has the **pointwise action** given by $\pi U = \{\pi u \mid u \in U\}$.

– The two-element set $\mathbb{B} = \{\top, \bot\}$ has a **trivial** permutation action such that $\pi x = x$ always.

We shall assume these permutation actions on these sets henceforth.

Given a predicate ϕ on atoms, write $\mathcal{N}a.\phi(a)$ (the Gabbay-Pitts **NEW quantifier** [19, 23]) for the assertion that:

– $\phi(a)$ is false for some finite set of atoms $a \in \mathbb{A}$ (which may be empty).

– $\phi(a)$ is true for all the other atoms.

Think of $\mathcal{N}a.\phi(a)$ intuitively as 'ϕ is true of *most* atoms'. For $a \in \mathbb{A}$ and $x \in \mathbb{X}$ write $a\#x$ when $\mathcal{N}b.(b\ a)x = x$. Read this as a **is fresh for** x. For example:

– For $\mathbb{X} = \mathbb{A}$, $a\#x$ when $a \neq x$.

– For $\mathbb{X} = \mathbb{L}$, $a\#[a_1, \ldots, a_n]$ if and only if $a \notin \{a_1, \ldots, a_n\}$.
– For $\mathbb{X} = \mathcal{P}_{fs}$, $a\#U$ when U is finite and $a \notin U$, or when U is cofinite and $a \in U$.
– For $\mathbb{X} = \mathbb{B}$, $a\#\top$ and $a\#\bot$ always.

Think of $a\#x$ intuitively as 'does not occur in any *distinguished* way'. The example $\mathcal{P}_{fs}$ shows that $a\#U$ does not mean $a \notin U$ — an atom may occur in U, but if *most* atoms occur in U, then $a\#U$.

If $\mathbb{X}$ is a set with a permutation action say $x \in \mathbb{X}$ is **finitely supported** when $\mathit{M}a.a\#x$. Write $supp(x)$ for the set $\{a \in \mathbb{A} \mid \neg a\#x\}$ and call this the **support** of x — $supp(x)$ is the set of 'atoms which occur in some distinguished way' in x. Say $\mathbb{X}$ has a **finitely supported** action when all its elements have finite support.

Call a set with a finitely supported permutation action a **nominal set** [23, 38]. Henceforth $\mathbb{X}$ and $\mathbb{Y}$ range over nominal sets.

– $\mathbb{A}$ is a nominal set and $supp(a) = \{a\}$.
– $\mathbb{L}$ is a nominal set and $supp([a_1, \ldots, a_n]) = \{a_1, \ldots, a_n\}$.
– $\mathcal{P}_{fs}$ is a nominal set and $supp(U) = U$ if U is finite, and $supp(U) = \mathbb{A} \setminus U$ if U is cofinite.
– $\mathcal{P}$ the set of *all* sets of atoms, is not a nominal set — it suffices to biject $\mathbb{A}$ with the natural numbers and use this to create a subset E of 'every other atom' — call this **the comb**. The comb does not have finite support.
– $\mathbb{B}$ is a nominal set and $supp(\top) = \emptyset = supp(\bot)$.

Call a function $f : |\mathbb{X}| \to |\mathbb{Y}|$ **equivariant** when $\pi(fx) = f(\pi x)$ always.

For any two nominal sets $\mathbb{X}$ and $\mathbb{X}'$ the set of all functions $f \in |\mathbb{X}| \to |\mathbb{X}'|$ has a permutation action, specified by $(\pi f)(\pi x) = \pi(f(x))$. This specifies πf because π is a bijection and it follows that $(\pi f)x = \pi(f(\pi^{-1}x))$.

This is not a nominal set because it has elements with infinite support. Take $\mathbb{X} = \mathbb{A}$ and $\mathbb{Y} = \mathbb{B}$; then the function δ_E such that $\delta_E(a) = \top$ if $a \in E$ and $\delta_E(a) = \bot$ if $a \notin E$, does not have finite support, where E is 'the comb' mentioned above.

Write $\mathbb{X} \Rightarrow \mathbb{Y}$ for the nominal set of functions $f \in |\mathbb{X}| \to |\mathbb{Y}|$ with finite support. As is standard $\Rightarrow$ associates to the right, for example $\mathbb{X} \Rightarrow \mathbb{Y} \Rightarrow \mathbb{Z}$ is $\mathbb{X} \Rightarrow (\mathbb{Y} \Rightarrow \mathbb{Z})$.

For any two nominal sets $\mathbb{X}$ and $\mathbb{X}'$ the set of pairs of elements $(x, x') \in |\mathbb{X}| \times |\mathbb{X}'|$ has a natural pointwise permutation action given by $\pi(x, x') = (\pi x, \pi x')$. It is easy to prove that $supp((x, x')) = supp(x) \cup supp(x')$, and so we obtain a nominal set $\mathbb{X} \times \mathbb{X}'$ in a natural way.

3 Substitution sets

Recall that we intend to give denotational semantics to variables. In the previous section we gave a semantics to *names*. But a variable is more than a name — a variable can be substituted for.

A **substitution set** is a nominal set $\mathbb{X}$ with:

– a equivariant **interpretation function** $[\![\text{-}]\!]_{\mathbb{X}}$ from $\mathbb{A}$ to $\mathbb{X}$, and
– an equivariant **substitution action** $\Sigma_{\mathbb{X}}$ from $\mathbb{X} \times \mathbb{A} \times \mathbb{X}$ to $\mathbb{X}$,

satisfying the following properties:

$$\begin{array}{rl}(\alpha) & b\#z \Rightarrow z[a{\mapsto}x] = ((b\ a)z)[b{\mapsto}x] \\ (\#{\mapsto}) & a\#z \Rightarrow z[a{\mapsto}x] = z \\ (\mathbf{var}{\mapsto}) & a[a{\mapsto}x] = x \\ (\mathbf{ren}{\mapsto}) & b\#z \Rightarrow z[a{\mapsto}b] = (b\ a)z\end{array}$$

If in addition the action satisfies

$$(\mathbf{sub}{\mapsto}) \quad a\#y \Rightarrow z[a{\mapsto}x][b{\mapsto}y] = z[b{\mapsto}y][a{\mapsto}x[b{\mapsto}y]]$$

then we call the substitution set **confluent**, otherwise we call it **non-confluent**. More on confluent versus non-confluent substitution sets later.

Informally we should read:

– $z[a{\mapsto}x]$ as 'z with a replaced by x';
– $a\#z$ as 'a does not occur free in z' (and $\Rightarrow$ as 'implies');
– $(b\ a)z$ as 'simultaneously rename b to a and a to b in z'.

We generally write $\Sigma_{\mathbb{X}}(z,a,x)$ as $z[a{\mapsto}x]$. We may write $[\![a]\!]_{\mathbb{X}}$ as $[\![a]\!]$, if $\mathbb{X}$ is understood or unimportant. We may even write $[\![a]\!]$ as a. For example $(\mathbf{var}{\mapsto})$ is more properly $[\![a]\!][a{\mapsto}x]$ and $(\mathbf{ren}{\mapsto})$ is more properly $z[a{\mapsto}[\![b]\!]]$. We may write $|\mathbb{X}|$ for the underlying set of the associated nominal set. We may write $x \in \mathbb{X}$ for $x \in |\mathbb{X}|$, and call $|\mathbb{X}|$ the **elements** of $\mathbb{X}$.

$\mathcal{Q}$. Where do these axioms come from? $\mathcal{A}$. We have proved them sound and complete [22] for a closed term model in a suitable and precise sense.

$\mathcal{Q}$. When is the substitution action uniquely defined on a nominal set, for a given interpretation function? $\mathcal{A}$. There are nominal sets with only one possible substitution action, for example $\mathbb{A}$ (the action is described below). However in the vast majority of cases many different substitution actions can be imposed on the same nominal set.

Example 3.1. 1. $\mathbb{A}$ is a substitution set if we interpret a by itself and interpret substitution by itself:

$$x[a{\mapsto}y] = y \quad (x = a) \qquad x[a{\mapsto}y] = x \quad (x \neq a).$$

2. $\mathbb{L}$ is a substitution set if we interpret a by the one-element list $[a]$ and we interpret substitution by list insertion. We omit a full definition but two examples should suffice:

$$[a,b,c][b{\mapsto}[a,b,c]] = [a,a,b,c,c] \qquad [a,a,b][b{\mapsto}[b,a]] = [a,a,b,a].$$

3. $\mathbb{L}$ is a substitution set if we interpret a by the one-element list $[a]$; we interpret substitution $m[a{\mapsto}l]$ by list insertion if $l = [a]$, if $l = [b]$ and b does not occur in m, or if a does not occur in m; otherwise $m[a{\mapsto}l] = []$. For example:

$$[a,b,c][a{\mapsto}[a]] = [a,b,c] \qquad [a,b,c][a{\mapsto}[b]] = [] \qquad [a,b,c][a{\mapsto}[d]] = [d,b,c]$$
$$[a,b,c][d{\mapsto}[a,b]] = [a,b,c] \qquad [a,b,c][c{\mapsto}[c,d]] = [].$$

This example is a counterexample for Lemma 3.2 part 1.

4. Consider $\mathbb{L}$ again where we interpret substitution by list insertion except if we try to substitute a list mentioning an atom already mentioned elsewhere in which case we default to the substituted value. For example:

$$[a,b,c][a{\mapsto}[b,c,d]] = [b,c,d] \qquad [a,b,c][a{\mapsto}[d]] = [d,b,c]$$
$$[a,b][a{\mapsto}[a]] = [a,b] \qquad [b,c][a{\mapsto}[b,c,d]] = [b,c].$$

   This example is a counterexample for Lemma 3.2 part 2.
5. On $\mathcal{P}_{fs}$ interpret an atom a by the singleton set $\{a\}$. For two sets $U, V \in \mathcal{P}_{fs}$ write

$$U \,\Delta\, V \;=\; \{a \mid (a \in U \wedge a \notin V) \vee (a \notin U \wedge a \in V)\}$$

   for the **symmetric difference** (**XOR**). Then interpret substitution by:

$$W[a{\mapsto}U] = W \quad \text{if } a\#W, \quad \text{and otherwise}\ldots \quad W[a{\mapsto}U] = (W \,\Delta\, \{a\}) \,\Delta\, U.$$

   The proof that this is a substitution set is by detailed calculations. This example is used to provide a counterexample in Lemma 5.5.

Substitution as we understand it from syntax is a cut-and-paste operation along the lines of the list insertion in part 2 of Example 3.1. The following result proves that our axioms are a little more general than that:

Lemma 3.2.

– *Suppose* $\neg a\#z$. *Then* $supp(z[a{\mapsto}x]) = (supp(z)\backslash\{a\}) \cup supp(x)$ *need* not *hold.*
– *There are non-confluent substitution sets.*

Proof. – Consider $\mathbb{L}$ with the second substitution action: then $supp([a,b][a{\mapsto}[b]]) = supp([]) = \emptyset$ and $(\{a,b\} \setminus \{a\}) \cup \{b\} = \{b\}$.
– Consider $\mathbb{L}$ with the third substitution action: then

$$[a,b,c][a{\mapsto}[c]][b{\mapsto}[b,c]] = [c] \quad \text{but} \quad [a,b,c][b{\mapsto}[b,c]][a{\mapsto}[c][b{\mapsto}[b,c]]] = [b,c] \neq [c]. \qquad \square$$

4 The environment model

Lemma 4.1. *If* $x \in \mathbb{X}$ *and* $f \in \mathbb{X} \to \mathbb{X}'$ *has finite support then* $supp(f(x)) \subseteq supp(f) \cup supp(x)$.

Proof. It suffices to show that $a\#f$ and $a\#x$ imply $a\#f(x)$. This is easy: for any $b\#f$ and $b\#x$, $(b\ a)(f(x)) = ((b\ a)f)((b\ a)x) = f(x)$. There are cofinitely many such b, and the result follows. $\square$

Recall that $\kappa \in \mathbb{A} \Rightarrow \mathbb{X}$ is a function on underlying sets with finite support; call these functions **environments**. Henceforth, κ varies over environments. As a matter of useful notation, when κ and κ' are such that $\kappa(a) = \kappa'(a)$ for all $a \in A$ we shall say that κ and κ' **agree on** A. It is easy to prove this result by calculation:

Lemma 4.2. *For a set $\mathbb{X}$ with the trivial permutation action, and any $f \in \mathbb{A} \to \mathbb{X}$, f is an environment (i.e. f has finite support) if and only if $И a.И b.f(a) = f(b)$.*

Fix some **set of base elements** $\mathbb{T}$ and endow it with the trivial permutation action. $\mathbb{T}$ could be $\mathbb{B} = \{\top, \bot\}$ or $\mathbb{N} = \{0, 1, 2, \ldots\}$.

Write $\hat{\mathbb{T}}$ for the set of $\tau \in (\mathbb{A} \Rightarrow \mathbb{T}) \Rightarrow \mathbb{T}$ such that if κ and κ' agree on $supp(\tau)$ then $\tau\kappa{=}\tau\kappa'$. Say τ has **no asymptotic behaviour**. Let τ, μ, ν vary over elements of $\hat{\mathbb{T}}$. Call this the **environment model of the set** $\mathbb{T}$.

For example:

1. $\tau\kappa{=}\kappa(a)$ has no asymptotic behaviour.
2. τ such that $И a.(\tau\kappa = \kappa(a))$ has asymptotic behaviour.
3. If $\mathbb{T} = \mathcal{P}_{fs}$ then $\tau\kappa = \{a \mid \kappa(a){=}\{\}\}$ has asymptotic behaviour.

Theorem 4.3. *If $\mathbb{T}$ is an 'ordinary set' then $\hat{\mathbb{T}}$ is a confluent substitution set with interpretation function $[\![a]\!] = \lambda\kappa.\kappa(a)$ and substitution action $(\nu[a{\mapsto}\tau])\kappa = \nu(\kappa\{a{\mapsto}\tau\kappa\})$.*

Here $\kappa\{a{\mapsto}\tau\kappa\}$ denotes the environment identical to κ except that a maps to $\tau\kappa$.

Proof. $\pi(\lambda\kappa.\kappa(a)) = \lambda\kappa.\kappa(\pi a)$ follows by detailed calculations. Similarly for substitution. $[\![a]\!][a{\mapsto}\tau]\kappa = (\kappa\{a{\mapsto}\tau\kappa\})(a) = \tau\kappa$. Suppose $a\#\tau'$. Then $\tau'[a{\mapsto}\tau]\kappa = \tau'(\kappa\{a{\mapsto}\tau\kappa\})$. From our assumption of no asymptotic behaviour, $\tau'(\kappa\{a{\mapsto}\tau\kappa\}) = \tau'\kappa$. Suppose $b\#\tau'$. Then $\tau'[a{\mapsto}[\![b]\!]]\kappa = \tau'(\kappa\{a{\mapsto}\kappa(b)\})$. Now $\tau'(\kappa\{a{\mapsto}\kappa(b)\}) = \tau'(\kappa\{a{\mapsto}\kappa(b)\}\{b{\mapsto}\kappa(a)\})$ since τ' has no asymptotic behaviour. By further calculations and recalling that $\mathbb{T}$ has the trivial action, $(\kappa\{a{\mapsto}\kappa(b)\}\{b{\mapsto}\kappa(a)\}) = (b\ a)\kappa$. By further calculations $\tau'[a{\mapsto}[\![b]\!]]\kappa = ((b\ a)\tau')\kappa$. (α) follows by concrete calculations. For $(\mathbf{sub}{\mapsto})$ suppose $a\#\nu$. Then

$$\tau[a{\mapsto}\mu][b{\mapsto}\nu]\kappa = \tau[a{\mapsto}\mu](\kappa\{b{\mapsto}\nu\kappa\}) = \tau(\kappa\{b{\mapsto}\nu\kappa\}\{a{\mapsto}\mu(\kappa\{b{\mapsto}\nu\kappa\})\}.$$

We then unfold $\tau[b{\mapsto}\nu][a{\mapsto}\mu[b{\mapsto}\nu]]\kappa$ in a similar way, rearrange, and use our assumptions of no asymptotic behaviour; the result follows by calculations. □

Example 4.4. Recall that $\mathbb{N}$ is the 'ordinary' set of natural numbers $\{0, 1, 2, 3, \ldots\}$. Consider $\hat{\mathbb{N}}$ and the expression $a * b$ — exactly the kind of thing which gets written on blackboards in school in algebra class.

This may be directly represented in $\hat{\mathbb{N}}$ as $\lambda\kappa.[\![a]\!]\kappa * [\![b]\!]\kappa$.

The reader can check that if we apply the substitution $[a{\mapsto}b + c]$ ($b + c$ is shorthand for $\lambda\kappa.[\![b]\!]\kappa + [\![c]\!]\kappa$) to $a * b$, then we obtain an element which may naturally be represented as $(b + c) * b$ — so we see in the semantics of variables (the interpretation of atoms) the same behaviour we naturally associate to variables in syntax.

5 Substitution on function-sets

If $\mathbb{X}$ and $\mathbb{Y}$ are (non-)confluent substitution sets then we can define a (non-)confluent substitution set $\mathbb{X} \times \mathbb{Y}$ with underlying set $|\mathbb{X}| \times |\mathbb{Y}|$ (the set of pairs (x, y) with $x \in \mathbb{X}$ and $y \in \mathbb{Y}$) and interpretation map $[\![a]\!]_{\mathbb{X}\times\mathbb{Y}} = ([\![a]\!]_{\mathbb{X}}, [\![a]\!]_{\mathbb{Y}})$, and substitution action $(x', y')[a{\mapsto}(x, y)] = (x'[a{\mapsto}x], y'[a{\mapsto}y])$. Validating equivariance and the axioms is very easy.

We need some technical machinery:

Lemma 5.1. *Suppose $\mathbb{X}$ is a (possibly non-confluent) substitution set, $x \in \mathbb{X}$, $z \in \mathbb{X}$, and $a\#x$. Then $a\#z[a{\mapsto}x]$ always.*

Proof. Choose some fresh b (so $b\#x, z, a$). It suffices to show that $(b\ a)(z[a{\mapsto}x]) = z[a{\mapsto}x]$. From $a\#x$ and $b\#x$ we can deduce that $(b\ a)x = x$. By equivariance it follows that $(b\ a)(z[a{\mapsto}x]) = ((b\ a)z)[b{\mapsto}x]$. Since $b\#z$ by (α) we have that $((b\ a)z)[b{\mapsto}x] = z[a{\mapsto}x]$. This suffices to establish the result. □

Recall that $h \in \mathbb{X} \Rightarrow \mathbb{Y}$ when $h \in |\mathbb{X}| \to |\mathbb{Y}|$ and h has finite support.

If $a \in \mathbb{A}$ write $[\![a]\!]$ for the function mapping $x \in \mathbb{X}$ to $[\![a]\!] \in \mathbb{Y}$. The support of $[\![a]\!]$ is $\{a\}$, which is finite, and it is a function from $|\mathbb{X}|$ to $|\mathbb{Y}|$, so $[\![a]\!] \in \mathbb{X} \Rightarrow \mathbb{Y}$.

Definition 5.2. *Fix $h, f \in \mathbb{X} \Rightarrow \mathbb{Y}$ and $a \in \mathbb{A}$. Specify the* **substitution action** *$h[a{\mapsto}f]$ by*

$$h[a{\mapsto}[\![n]\!]]x = ((n\ a)h)x \qquad \textit{If } n \in \mathbb{A} \textit{ and } n\#h \textit{ or } n{=}a$$
$$(h[a{\mapsto}f])x = (((a'\ a)h)x)[a'{\mapsto}fx] \qquad \textit{Otherwise, and any } a'\#h, a, f, x.$$

Recall that $((b\ a)h)x = (b\ a)(h((b\ a)x))$.

Lemma 5.3. *It does not matter* which *a' we choose to calculate $((a'\ a)h)x)[a'{\mapsto}fx]$ in the second case of the definition of $h[a{\mapsto}f]$, provided that $a'\#h, a, f, x$.*

Proof. $a'\#(fx \in \mathbb{Y})$ by Lemma 4.1. By Lemma 5.1 the result follows. □

It is easy to verify that the interpretation function and substitution actions are equivariant on $\mathbb{X} \Rightarrow \mathbb{Y}$.

Theorem 5.4. *With these definitions, $X \Rightarrow Y$ is a substitution set. Concretely that means:*

- *Substitution is equivariant: $(b\ a)(h[n{\mapsto}f]) = ((b\ a)h)[(b\ a)n{\mapsto}(b\ a)f]$ always.*
- *$\mathbb{X} \Rightarrow \mathbb{Y}$ satisfies (α); if $c\#h$ then $h[a{\mapsto}f] = ((c\ a)h)[c{\mapsto}f]$ always.*
- *$\mathbb{X} \Rightarrow \mathbb{Y}$ satisfies $(\#{\mapsto})$; if $a\#h$ then $h[a{\mapsto}f] = h$.*
- *$\mathbb{X} \Rightarrow \mathbb{Y}$ satisfies $(\mathbf{var}{\mapsto})$ and $(\mathbf{ren}{\mapsto})$.*

Proof. Equivariance is by detailed calculations which we cannot include here. For the rest, there are two cases.

Suppose $f = [\![b]\!]$ for $b\#h$. Then $(\mathbf{ren}{\mapsto})$ is immediate since $h[a{\mapsto}[\![b]\!]] = (b\ a)h$ by definition. For (α) suppose $c\#h$. There are two cases: If $c = b$ then $((b\ a)h)[b{\mapsto}[\![b]\!]] = h$ and $(b\ b)h = h$ and we are done. If $c\#b$ then $((c\ a)h)[c{\mapsto}[\![b]\!]] = (b\ c)(c\ a)h$ and simple calculations verify that if $c\#h$ then $(b\ c)(c\ a)h = (b\ a)h$. If $a\#h$ then $h[a{\mapsto}[\![b]\!]] = (b\ a)h = h$ so $(\#{\mapsto})$ follows. Finally for $(\mathbf{var}{\mapsto})$ if $h = [\![a]\!]$ then $[\![a]\!][a{\mapsto}[\![b]\!]] = (b\ a)[\![a]\!] = [\![b]\!]$ is easily checked.

Otherwise, by extensionality it suffices to check equality applied to $x \in \mathbb{X}$. Choose $x \in \mathbb{X}$ and $a'\#h, a, f, x$. Then $(h[a{\mapsto}f])x = (((a'\ a)h)x)[a'{\mapsto}fx]$. For $(\mathbf{ren}{\mapsto})$ there is nothing to prove. (α) follows by calculations. If we suppose $a\#h$ then $(a'\ a)h = h$. By Lemma 4.1 also $a\#hx$ and by $(\#{\mapsto})$ for $\mathbb{Y}$ we have $(((a'\ a)h)x)[a'{\mapsto}fx] = hx$ and $(\#{\mapsto})$ follows. Finally for $(\mathbf{var}{\mapsto})$ if $h = [\![a]\!]$ then

$$(((a'\ a)[\![a]\!]_{\mathbb{X}\Rightarrow\mathbb{Y}})x)[a'\mapsto[\![b]\!]_{\mathbb{X}\Rightarrow\mathbb{Y}}x] = ([\![a']\!]_{\mathbb{Y}})[a'\mapsto[\![b]\!]_{\mathbb{Y}}] = [\![b]\!]_{\mathbb{Y}} = [\![b]\!]_{\mathbb{X}\Rightarrow\mathbb{Y}}x. \qquad \square$$

Here we use our convention from Section 3 where the subscript $[\![a]\!]_{\mathbb{X}}$ indicates the substitution set to which the interpretation function belongs.

Q. In nominal sets only equivariant *functions are considered.* $\mathcal{A}$. Equivariant functions are *arrows* between sets in the category of nominal sets. Plenty of non-equivariant functions exist as elements of objects; our $\mathbb{X} \Rightarrow \mathbb{Y}$ is the exponential of the underlying category of nominal sets and the functions above live there.

Lemma 5.5. *Even if* $\mathbb{X}$ *and* $\mathbb{Y}$ *are confluent,* $\mathbb{X} \Rightarrow \mathbb{Y}$ *might be non-confluent.*

Proof. It suffices to provide a counterexample. Let $\mathbb{X} = \mathbb{Y} = \mathcal{P}_{fs}$ with the substitution action from Section 3. Take $h = \lambda U.U \cup \{a, b\}$ and $f = \lambda U.U \cup \{b, c\}$. Let $g = \lambda U.\{c\}$ (that is, $g = [\![c]\!]$). Then $a\#g$ and

$$\begin{aligned}
h[a\mapsto f][b\mapsto g]\{c\} &= ((b'\ b)(h[a\mapsto f]\{c\}))[b'\mapsto g\{c\}] \\
&= ((b'\ b)\big(((a'\ a)h)\{c\}[a'\mapsto f\{c\}]\big))[b'\mapsto g\{c\}] \\
&= \{a', b', c\}[a'\mapsto\{b', c\}][b'\mapsto\{c\}] \\
&= (\{b', c\}\ \Delta\ \{b', c\})[b'\mapsto\{c\}] \\
&= \{\}[b'\mapsto\{c\}] = \{\} \\
h[b\mapsto g][a\mapsto f[b\mapsto g]]\{c\} &= ((c\ b)h)[a\mapsto f[b\mapsto g]]\{c\} = \cdots \\
&= \{a', c\}[a'\mapsto\{b', c\}[b'\mapsto\{c\}]] = \{c\}. \qquad \square
\end{aligned}$$

Q. How do we know that this is the 'right' substitution action? $\mathcal{A}$. We do not: Definition 5.2 appears to be what is necessary to make Theorem 5.4 work; there may be other definitions and we do not yet have an overview of the design space.

Q. How dare we inflict a non-canonical *definition upon the reader?* $\mathcal{A}$. No single modal or temporal logic is canonical; there is a family whose members are useful in linguistics, hardware verification, and much more besides. No single notion of truth is canonical either; truth may be classical, intuitionistic, relevant, and so on. The π-calculus is non-canonical but useful, and so are the different implementations of ordered pairs in set theory. So our substitution action is non-canonical; it lets us express what we want to express, so what?

Q. Perhaps we should be really *cunning and design an alternative definition of substitution which is confluent also on function-spaces.* $\mathcal{A}$. No, we should not! Recall that our motivation is to permit programming on variables as names, i.e. as a form of data (a *very special* form of data, with a substitution action). Confluence effectively guarantees that we can do nothing with a variable that we cannot do with an instantiation of it, or put another way, a confluent substitution set cannot 'see' the names of variables in it. Lemma 5.5 makes this point formally, by proving that even if $\mathbb{X}$ and $\mathbb{Y}$ cannot see their own variables in this sense, the functions between them certainly may. To wish to guarantee confluence is to be blind to the motivation of this paper.

Q. So where is the programming language to go with this semantics? $\mathcal{A}$. There is no *one* programming language to go with a semantics of this style, and even if there were we would have no space to develop it here. The closest we can reasonably get here and now is the discussion in the next section.

6 Examples

In the Introduction we promised that our semantics would allow us to express interesting new functions in which variables are data. Here are some examples; many others are possible.

- If $x \in \mathbb{X}$ then 'the substitution $[a{\mapsto}x]$' exists as $\lambda z.z[a{\mapsto}x] \in \mathbb{X} \Rightarrow \mathbb{X}$.
- The map $\lambda x.\lambda x'.x[a{\mapsto}x'] \in \mathbb{X} \Rightarrow (\mathbb{X} \Rightarrow \mathbb{X})$ is a function by which every atom a defines a mapping from $\mathbb{X}$ to $\mathbb{X} \Rightarrow \mathbb{X}$. Elements of substitution sets are functions over their peers, defined by substitution for an atom. We should think of $\lambda x.\lambda x'.x[a{\mapsto}x']$ as '$(\lambda a.\text{-})$-'.
  There is no sense in which $x \in \mathbb{X}$ has 'higher type' then $x' \in \mathbb{X}$ and we are free to write $x[a{\mapsto}x']$ or $x'[a{\mapsto}x]$, or both, as in the function $\lambda x.\lambda x'.(x[a{\mapsto}x'], x'[a{\mapsto}x]) \in \mathbb{X} \Rightarrow \mathbb{X} \Rightarrow (\mathbb{X} \times \mathbb{X})$.
- The map $\lambda n.\lambda x.\lambda x'.x[n{\mapsto}x'] \in \mathbb{A} \Rightarrow \mathbb{X} \Rightarrow \mathbb{X} \Rightarrow \mathbb{X}$ abstracts over the free atom in the previous example. Here we see an atom being used both as data (the argument of a function) and as a variable (since it is explicitly substituted for).
  We can do case-splits on equality of atoms, as in the function

$$\lambda n.\lambda n'.\lambda x.\lambda x'.\begin{cases}(x[n{\mapsto}x'], x'[n'{\mapsto}x]) & n \neq n' \\ (x, x') & n = n'\end{cases} \in \mathbb{A} \Rightarrow \mathbb{A} \Rightarrow \mathbb{X} \Rightarrow \mathbb{X} \Rightarrow (\mathbb{X} \times \mathbb{X}).$$

  We do not recommend that the reader use this particular function in daily life but it illustrates our two uses of atoms: as data which can be compared for equality, and as variables which can be instantiated by an explicit substitution.
- Consider $a_{\mathbb{X}\Rightarrow\mathbb{Y}}(a_{\mathbb{X}})$. Expanding definitions, $a_{\mathbb{X}\Rightarrow\mathbb{Y}} = \lambda x_{\mathbb{X}}.a_{\mathbb{Y}}$, so $a_{\mathbb{X}\Rightarrow\mathbb{Y}}(a_{\mathbb{X}}) = a_{\mathbb{Y}}$. Indeed $a_{\mathbb{X}\Rightarrow\mathbb{Y}}(x) = a_{\mathbb{Y}}$ *always*. Consequently, $(a_{\mathbb{X}\Rightarrow\mathbb{Y}}(x))[a{\mapsto}y_{\mathbb{Y}}] = y$ always and we should not think of $a_{\mathbb{X}\Rightarrow\mathbb{Y}}(a_{\mathbb{X}})$ as 'a applied to itself', but simply as 'the a in $\mathbb{Y}$'.
- Recall that $\mathbb{B} = \{\top, \bot\}$ is a two-element 'ordinary set'. There is a function $\# \in (\mathbb{A}\times\mathbb{X}) \Rightarrow \hat{\mathbb{B}}$ ($\hat{\mathbb{B}}$ is notation from Section 4) such that $\#(a, x) = \lambda\kappa.\top$ when $a\#x$ and $\#(a, x) = \lambda\kappa.\bot$ otherwise. This function explicitly checks whether an atom a occurs free in x.
- The intensional $\equiv$ and extensional $=$ equalities on $\hat{\mathbb{N}}$ mentioned in the Introduction can easily be expressed as functions $(x \equiv y)\kappa$ is $\top$ if x is equal to y and $\bot$ otherwise, and $(x = y)\kappa$ is $\top$ if $x\kappa = y\kappa$ and $\bot$ otherwise; both functions are in $\hat{\mathbb{N}} \Rightarrow \hat{\mathbb{N}} \Rightarrow \hat{\mathbb{B}}$.
- An 'exception handler' is $\lambda x.\texttt{if}\ a\#x\ \texttt{then}\ x\ \texttt{else}\ x' \in \mathbb{X} \Rightarrow \mathbb{X}$. This treats a as an exception which if detected defaults to x'. It is important to appreciate that exceptions can be implemented by adjoining elements to the result type $\mathbb{X}$, as well as the types of all intermediate steps between the result and the site where the exception may be raised, and adding case-splits to catch the exception and propagate it correctly (see [28] for an excellent description of how this *can* be tamed).
  Here we are doing something different; a is propagated *for us* in a suitable sense.

Relation to nominal sets, and some categorical language Nominal sets were described in Section 2; a substitution set is just a nominal set with some extra structure (an interpretation function and substitution action; see Section 3). Recall the definition of *equivariant* from Section 2.

So what does the category of substitution sets look like? Let $\mathbb{S}$ be the category with objects substitution sets and arrows from $\mathbb{X}$ to $\mathbb{Y}$ equivariant functions between the underlying nominal sets. Write the set of arrows $\mathbb{X} \longrightarrow \mathbb{Y}$.

Lemma 6.1. *$\mathbb{S}$ is categorically equivalent to the category of* nonempty nominal sets.

Proof. The equivalence is given by the map to the underlying nominal set, and in the other direction by an elaboration of the environment model from Section 4, generalised to the case that $\mathbb{T}$ is a nominal set.

$\mathcal{Q}$. Since the arrows of $\mathbb{S}$ are just arrows in the underlying category of nominal sets, Lemma 6.1 is uninteresting. $\mathcal{A}$. Quite so, but people keep asking.

$\mathcal{Q}$. $\mathbb{S}$ is wrong; arrows should be structure-preserving *morphisms and so we should require of an arrow h that $h(z[a{\mapsto}x]) = (hz)[a{\mapsto}hx]$.* $\mathcal{A}$. No, no, and again no! To insist that $h(z[a{\mapsto}x]) = (hz)[a{\mapsto}hx]$ is to insist that h can do nothing to a variable as a *name*; in effect we sneak confluence in through the back door — see the discussion of confluence towards the end of Section 5.

$\mathcal{Q}$. $\mathbb{S}$ is just nominal sets with extra structure on the objects which is not reflected in the arrows. This makes $\mathbb{S}$ useless for studying any aspect of the substitution structure. $\mathcal{A}$. This makes *categorical techniques* useless for studying any aspect of the substitution structure. We are using other methods.

$\mathcal{Q}$. Categorical treatments of substitution exist; consider [18, 17], which (translated into the language of this paper) amount to properties $a[a{\mapsto}x] = x$, $x[a{\mapsto}a] = x$, and $z[a{\mapsto}x][b{\mapsto}y] = z[b{\mapsto}y][a{\mapsto}x[b{\mapsto}y]]$ if $a\#y$. What about them? $\mathcal{A}$. The third property is confluence again. We care about a semantics that supports functions that program on the names of variables. The work cited is unsuitable to this task in the precise sense that with confluence, it is not possible to directly interpret the examples of the previous section as arrows. More subtly, the papers [18, 17] use a presheaf category *without* the condition that presheaves preserve pullbacks of pairs of monos. Translated into our language this means that there is no single least supporting set of names for an element (we are not aware of this observation having been made previously in the literature). Unique least supporting sets are a significant practical convenience, at least in our experience.

Valuation-free semantics of Λ. We promised the reader one at the end of the Introduction. Fix some type environment Σ. Let $o = \hat{\mathbb{B}}$ and let $\tau \to \tau' = \tau \Rightarrow \tau'$ (we silently overload types and the substitution sets representing them, just as we did in the Introduction). Then

$$[\![a]\!] = a_{\Sigma(a)} \qquad [\![\lambda a{:}\tau.t]\!] = \lambda x \in \tau.[\![t]\!][a{\mapsto}x] \qquad [\![tt']\!] = [\![t]\!]([\![t']\!]).$$

Theorem 6.2. *If $\Sigma \vdash t : \tau$ then $[\![t]\!]$ is well-defined and $[\![t]\!] \in \tau$.*

Proof. We work by induction on t.

- By construction $\Sigma \vdash a : \tau$ when $\Sigma(a) = \tau$ and since also by construction $[\![a]\!] = a_{\Sigma(a)}$ the result is immediate.

– Suppose $\Sigma \vdash \lambda a{:}\tau_2.t_1 : \tau_2 \to \tau_1$ (this is the only kind of typing a term of this form can have). Then $\Sigma, a{:}\tau_2 \vdash t_1 : \tau_1$. By inductive hypothesis $[\![t_1]\!]$ is well-defined and $[\![t_1]\!] \in \tau_1$. By the constructions above, we see that $\lambda x \in \tau_2.[\![t_1]\!][a \mapsto x] \in \tau_2 \Rightarrow \tau_1$ as required.
– The case of application is easy.

There are more things to check, of course, but they all work out. There are also many more functions in our semantics than represented above. It remains to study languages to represent them.

$\mathcal{Q}$. We would expect a 'substitution set' to be a type-indexed family (X_σ) with substitution a function of the form $X_\sigma \times \mathbb{A} \times Y_{\sigma'} \to X_\sigma$. $\mathcal{A}$. This is quite true; we have simplified. For applications a richer structure is useful and we have considered this — but it adds no essential difficulties. Our main point as always is to demonstrate functional programming on names of variables and we use the simplest possible setting to begin with.

7 Conclusions and related work

A variable has two properties; a name, and the ability to represent an unknown element. Typically, the name of a variable is visible in the syntax of a programming language, but not in its semantics. The transition between these two aspects of the notion of 'variable' is traditionally managed by *valuations* [42].

The denotation in this paper admits a direct representation of variables in the denotation as names-with-a-substitution-action. This makes valuations 'go away', but more importantly it allows us to give denotations to functions which reflect on the names of variables; we have in mind functions like substitution, λ-abstraction, and case-splits on intensional equality of variables (their names), occurrences of variables in elements, functions which bind variables in their arguments, and so on.

Our immediate future work is to test our semantics on Curry [24] and on a recent language with pattern-matching [3].

In 1999 the first author presented FM sets [23] in work with Pitts. This provided a semantics for α-conversion which in time gave rise to useful logics [8] and programming languages [41, 9]. We hope that the semantics of this paper will inspire further logics and programming languages, this time with built-in principles for explicit manipulation of their own variables.

More questions are: $\mathcal{P}_{fs}$ can easily be made into a semantics of classical propositional logic with quantifiers, can it be patched for the intuitionistic case [36]? Can we investigate semantics of the *untyped* λ-calculus by considering reflexive substitution-function-sets [32]? As mentioned at the end of the previous section, we should generalise substitution from $\mathbb{X} \times \mathbb{A} \times \mathbb{X} \to \mathbb{X}$ to $\mathbb{X} \times \mathbb{A} \times \mathbb{Y} \to \mathbb{X}$ for various $\mathbb{Y}$, in the style perhaps of [17] and thus obtain novel semantics of systems like predicate logic [25].

The issue of 'extension' versus 'intension' is well-known in philosophy and logic [2], and also well-studied in computer-science [26]. The typical example of such a development is 'how do we distinguish between two different proofs of the same proposition', or 'how do we distinguish between two algorithms that compute the same function'; variables are treated as extensionally as always. So far as we know the only intensional study of variables in logic, is a-logic [21].

So far as we know, the only treatments of substitution in a context liable to give semantics to functional programming come from category theory [18, 17]. As we have discussed above, these still treat variables extensionally and are not suitable to program on the name of a variable.

Treatments exist of substitution in the absence of functions. Crabbé gives axioms [13, 12] in first-order logic enriched with (in our terminology) atoms and freshness conditions. The similarities between the two axiomatisations is striking though there are also significant differences: for us atoms and freshness conditions are part of an ambient 'nominal framework' whose ideas (equivariance, abstractions, support, permutation action on functions) we have fully exploited to define the substitution action on functions, which Crabbé does not do. Crabbé does not consider functions ('...we are not concerned with the notion of bound variable' [12, page 2]') whereas they are the focus of our enquiry here. Crabbé also adheres to confluence (rule $(\mathbf{sub}{\mapsto})$, Crabbé's axiom 4) as 'the fundamental axiom of substitution' [12, page 3] whereas we have repeatedly argued in this paper that for our purposes, this is the axiom to drop!

Feldman [14] gives an algebraic axiomatisation (our axioms are *nominal* algebraic [22]). He is inspired by a concrete model of substitution similar to the environment model in Section 4 (we got the construction independently from conversations with Gilles Dowek). Feldman's axioms are distant from ours being closer in spirit to Cylindric Algebras [7] and Lambda Abstraction Algebras [31, 39].

Mariño [34] gives a semantics for Curry based on our premise in this paper that variables should exist in denotation. The constructions seem to be concrete instances of the constructions in this paper, without the accompanying theory — but this remains to be checked.

Combinatory Algebra [5], Illative Combinatory Logic [4] (based on combinatory algebra), and Predicate Functor Logic [29], attain the effect of substitution by emulations. For example see the translation of λ-terms into combinatory algebra terms [5]. Thanks to the use of combinators, variables just 'go away'.

We must not forget λ-calculi with explicit substitutions [1]. Lescanne's survey [30] and the thesis of Bloo [6] chart a vast literature on the topic. These decompose β-reduction as a rule to introduce explicit substitution ($(\lambda a.u)t \rightarrow u[a{\mapsto}t]$), and explicit rules for that substitution's subsequent behaviour (which is to substitute, of course). These calculi are designed to measure the cost of a β-reduction and do not directly reflect on variables' names; for example it is not possible to represent a function which case-splits on whether a variable is bound or not in its argument, or to represent the abstraction $\lambda a.$- directly, as we do.

This work benefits greatly from research into nominal techniques. The axiomatisation of substitution we use is inherited from work on nominal algebra [22]. This is descended from but not to be confused with previous less powerful axiomatisations in nominal rewriting [16, 15], itself based on nominal terms and unification [44]. Nominal sets go back to [23].

The NEW calculus of contexts (NEWcc) [20] is a λ-calculus with constructs which can be partially interpreted in the substitution-sets-and-functions of this paper. The NEWcc has a richer structure including a hierarchy of atoms, and is also weaker structure in that thus far we have not considered case-splits in the NEWcc on equality of names, nor explicit representation of freshness. Transfer of ideas is possible in both directions.

We see some connection with pointers, which we have yet to make formal. A pointer has some of the properties of a variable in that it represents an unknown element by *pointing to* it.

A pointer also has a name (itself). Pointers are incredibly useful, but also they are notoriously hard to work with. We would like to investigate to what degree our atoms are a semantics for 'well-behaved pointers' — note that this is entirely orthogonal to monadic semantics for state, which introduces controlled doses of sequentiality [33] into pure functional programming but which is not specific to state.

The semantics of most languages with variables use valuations. This is a formal way of saying 'a variable is an unknown denotational element and *nothing but* an unknown denotational element'. So subliminally is this notion introduced that we may forget to imagine alternatives. We say: awake! Variables can be so much more!

References

[1] M. Abadi, L. Cardelli, P.-L. Curien, J.-J. Lévy. Explicit substitutions. In *Conference Record of the Seventeenth Annual ACM Symposium on Principles of Programming Languages, San Francisco, California*. ACM, 1990.

[2] A. Anderson. General intensional logic. In F. Guenthner, D. Gabbay, eds., *Handbook of Philosophical Logic*, vol. II: Extensions of Classical Logic. Reidel, 1984.

[3] A. Arbiser, A. Miguel, A. Ríos. A lambda-calculus with constructors. In *17th International Conference on Rewriting Techniques and Applications*, 2006.

[4] H. Barendregt, W. Dekkers, M. Bunder. Completeness of two systems of illative combinatory logic for first-order propositional and predicate calculus. *Archive für Mathematische Logik*, 37, 1998.

[5] H. P. Barendregt. *The Lambda Calculus: its Syntax and Semantics (revised ed.)*, vol. 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, 1984.

[6] R. Bloo. *Preservation of Termination for Explicit Substitution*. PhD thesis, Eindhoven University of Technology, Eindhoven, 1997.

[7] S. Burris, H. Sankappanavar. *A Course in Universal Algebra*. Springer, 1981. Available online.

[8] L. Caires, L. Cardelli. A spatial logic for concurrency (part II). Extended version, with proofs, 2003.

[9] J. Cheney, C. Urban. Alpha-prolog: A logic programming language with names, binding and alpha-equivalence. In B. Demoen, V. Lifschitz, eds., *Proceedings of the 20th International Conference on Logic Programming (ICLP 2004)*, number 3132 in LNCS. Springer-Verlag, 2004.

[10] CIAO. CIAO Prolog. `www.clip.dia.fi.upm.es/Software/Ciao/ciao_html/ciao_toc.html`.

[11] H. Cirstea, L. Liquori, B. Wack. Rewriting calculus with fixpoints: Untyped and first-order systems. vol. 3085. Springer, 2003.

[12] M. Crabbé. On the notion of substitution. *Logic Journal of the IGPL*, 12 n.2, 2004.

[13] M. Crabbé. Une axiomatisation de la substitution. *Comptes rendus de l'Académie des Sciences de Paris, Série I*, 338, 2004.

[14] N. Feldman. Axiomatization of polynomial substitution algebras. *Journal of Symbolic Logic*, 47(3), 1982.

[15] M. Fernández, M. J. Gabbay. Nominal rewriting. *Information and Computation*, 2005. In press.

[16] M. Fernández, M. J. Gabbay, I. Mackie. Nominal rewriting systems. In *Proc. 6th Int. ACM SIGPLAN Conf. on Principles and Practice of Declarative Programming (PPDP'2004)*. ACM, 2004.

[17] M. Fiore, D. Turi. Semantics of name and value passing. In *Proc. 16th LICS Conf.* IEEE, Computer Society Press, 2001.

[18] M. P. Fiore, G. D. Plotkin, D. Turi. Abstract syntax and variable binding. In *14th Annual Symposium on Logic in Computer Science*. IEEE Computer Society Press, Washington, 1999.

[19] M. J. Gabbay. *A Theory of Inductive Definitions with alpha-Equivalence*. PhD thesis, Cambridge, UK, 2000.
[20] M. J. Gabbay. A new calculus of contexts. In *Proc. 7th Int. ACM SIGPLAN Conf. on Principles and Practice of Declarative Programming (PPDP'2005)*. ACM, 2005.
[21] M. J. Gabbay, M. J. Gabbay. a-logic. In *We Will Show Them: Essays in Honour of Dov Gabbay*, vol. 1. College Publications, 2005.
[22] M. J. Gabbay, A. Mathijssen. Capture-avoiding substitution as a nominal algebra. In *ICTAC'2006*, 2006.
[23] M. J. Gabbay, A. M. Pitts. A new approach to abstract syntax with variable binding. *Formal Aspects of Computing*, 13(3–5), 2001.
[24] M. Hanus, S. Antoy, H. Kuchen, F. J. López-Fraguas, W. Lux, J. J. Moreno-Navarro, F. Steiner. Curry: An integrated functional logic language. 2003.
[25] W. Hodges. Elementary predicate logic. In D. Gabbay, F. Guenthner, eds., *Handbook of Philosophical Logic, 2nd Edition*, vol. 1. Kluwer, 2001.
[26] M. Hofmann. *Extensional Constructs in Intensional Type Theory*. Springer, 1997.
[27] C. B. Jay. The pattern calculus. *ACM Transactions in Programming Language Systems*, 26(6), 2004.
[28] P. Jones. Tackling the awkward squad: monadic input/output, concurrency, exceptions, and foreign-language calls in haskell. In *Engineering theories of software construction, Marktoberdorf Summer School*, NATO ASI. 105 Press, 2001.
[29] S. T. Kuhn. An axiomatization of predicate functor logic. *Notre Dame Jounal Formal Logic*, 24(2), 1983.
[30] P. Lescanne. From lambda-sigma to lambda-upsilon a journey through calculi of explicit substitutions. In *POPL '94: Proc. 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM Press, 1994.
[31] S. Lusin, A. Salibra. The lattice of lambda theories. *Journal of Logic and Computation*, 14 n.3, 2004.
[32] A. R. Meyer. What is a model of the lambda calculus? *Information and Control*, 1(52), 1982.
[33] E. Moggi. Notions of computation and monads. *Inf. Comput.*, 93(1), 1991.
[34] J. M. no. *Semantics and Analysis of Functional Logic Programs*. PhD thesis, Universidad Politécnica de Madrid, Facultad de Informática, 2002.
[35] L. Paulson. *ML for the Working Programmer*. CUP, 2 edition, 1996.
[36] A. M. Pitts. On an interpretation of second order quantification in first order intuitionistic propositional logic. *Jour. Symbolic Logic*, 57, 1992.
[37] A. M. Pitts. Operationally-based theories of program equivalence. In P. Dybjer, A. M. Pitts, eds., *Semantics and Logics of Computation*, Publications of the Newton Institute. Cambridge University Press, 1997.
[38] A. M. Pitts. Nominal logic, a first order theory of names and binding. *Information and Computation*, 186(2), 2003.
[39] A. Salibra. On the algebraic models of lambda calculus. *Theoretical Computer Science*, 249(1), 2000.
[40] M. Sato, T. Sakurai, Y. Kameyama. A simply typed context calculus with first-class environments. *Journal of Functional and Logic Programming*, 2002(4), 2002.
[41] M. R. Shinwell, A. M. Pitts, M. J. Gabbay. FreshML: Programming with binders made simple. In *Eighth ACM SIGPLAN International Conference on Functional Programming (ICFP 2003), Uppsala, Sweden*. ACM Press, 2003.
[42] A. Tarski. The semantic conception of truth and the foundations of semantics. *Philosophy and Phenomenological Research*, 4, 1944.
[43] S. Thompson. *Haskell: The Craft of Functional Programming*. Addison Wesley, 1996.
[44] C. Urban, A. M. Pitts, M. J. Gabbay. Nominal unification. *Theoretical Computer Science*, 323(1–3), 2004.